

协程技术调研

从并发演进到底层原理·语言对比·实验验证

kyc

计算机学院

2026 年 6 月

一句话

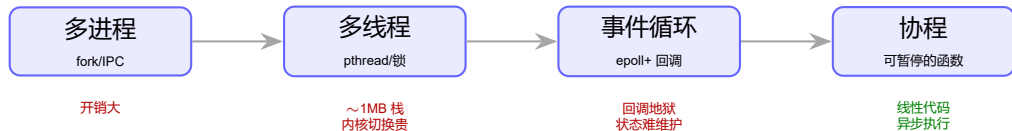
协程不是把 CPU 计算变快的魔法，而是用更低的内存和调度成本处理**大量等待中的 I/O 任务**。

- ❶ **Why**: 进程/线程解决并发，但在 C10K/C10M 规模下被栈空间和内核调度成本限制。
- ❷ **How**: 协程把回调式事件循环包装成可暂停函数，底层依赖编译器状态机、运行时调度器和 OS I/O 多路复用。
- ❸ **When**: I/O 密集、高并发、等待多时值得用；CPU 密集仍应依赖多核并行、SIMD 或 GPU。

目录

- 1 为什么需要协程：并发演进
- 2 协程的定义、分类与心智模型
- 3 底层机制：有栈 vs 无栈
- 4 三大应用范式
- 5 主流语言协程支持对比
- 6 编译器 × 运行时 × 操作系统
- 7 实验：协程 vs 多线程
- 8 选型建议与总结
- 9 AI 使用说明
- 10 参考资料

并发模型的演进



- **线程让步于规模**: C10K $\rightarrow$ C10M, 线程数撞到内存/调度天花板
- **回调让步于可读性**: epoll + callback 性能好, 但代码反人类
- **协程是「鱼与熊掌」的妥协**: 线性的代码 + 异步的执行

四种并发模型对比

维度	进程	线程	事件循环	协程
调度方	OS 抢占	OS 抢占	用户态	用户态 (协作)
切换开销	数十 μ s	1–5 μ s	$\sim$ 10 ns	10–100 ns
栈空间	MB 级	$\sim$ 1 MB	无 (堆闭包)	2–8 KB
编码风格	IPC 复杂	线性 + 锁	嵌套回调	线性 + await
典型规模	百级	千级	万–十万	百万级
适合场景	强隔离	CPU 密集	底层网络	I/O 密集

一句话总结

协程 = 用户态调度的、可暂停-恢复的函数，把「事件循环 + 状态机」从手写变成编译器/运行时生成。

协程是什么：把 callback 还原为函数

协程视角

线性、可读

```
1 Task<Response> handle(Request req) {  
2   auto user = co_await db.query(uid);  
3   auto data = co_await rpc.fetch(user);  
4   co_return render(data);  
5 }
```

回调视角

嵌套、难维护

```
1 def handle(req, cb):  
2   db.query(uid, lambda user:  
3     rpc.fetch(user, lambda data:  
4       cb(render(data))))
```

关键认知

协程不是「更快」，是把隐式的状态机变成显式由编译器生成。运行时复杂度不变，开发心智复杂度大大降低。

协程的两个分类维度

是否有独立栈

有栈协程 (Stackful)

- 每个协程一段独立栈
- 切换 = 切寄存器 + SP
- 任意深度都能 yield
- 代表: **Go**、**Lua**

无栈协程 (Stackless)

- 编译为状态机
- 局部变量存到堆上 frame
- 只能在顶层 await
- 代表: **C++20**、**Python**、**C#**

调度对称性

对称协程

- 协程之间互相 yield
- 类似 goroutine

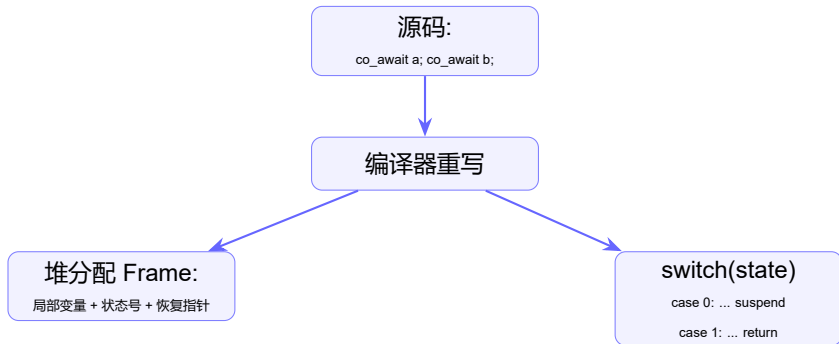
非对称协程

- 有明确的 caller/callee
- yield 必回到调用者
- 类似 generator、async/await

主流趋势

非对称 + 无栈 + 调度器驱动
更省内存, 更利于编译器优化

无栈协程：编译器把函数变成状态机



优点

- 无需独立栈
- 单协程内存 ≈ 几十-几百字节
- 切换 ≈ 一次间接跳转

缺点

- 必须由编译器改写
- co_await 不能嵌进任意函数深处
- C++20：标准库无调度器/Task/IO

有栈协程：栈切换的约 20 行汇编

核心操作：保存 callee-saved 寄存器 → 切 RSP → 恢复寄存器 → ret

x86_64 协程切换 (简化)

```
1 switch_ctx:
2     push    rbp
3     push    rbx
4     push    r12-r15      ; callee-saved
5     mov     [rdi], rsp    ; save old SP
6     mov     rsp, [rsi]    ; switch SP
7     pop     r15-r12
8     pop     rbx
9     pop     rbp
10    ret                ; jump to new
```

	有栈	无栈
切换成本	20–100 ns	5–20 ns
内存	KB 级	字节级
实现	汇编 + 栈管理	编译器改写
代表	Go, ucontext	C++20, Rust

应用范式: async/await · CSP · 结构化并发

1. async/await

非对称、无栈
与 Future/Task 协作

```
1 async def fetch():  
2     r = await http.get(url)  
3     return r.json()  
4  
5
```

C++20 / Python / C# / Rust

2. CSP / Channel

对称、有栈
通过 channel 通信

```
1 ch := make(chan int)  
2 go func() {  
3     ch <- compute()  
4 }()  
5 v := <-ch
```

Go, Kotlin

3. 结构化并发

父协程持有子协程生命周期

```
1 coroutineScope {  
2     val a = async { f1() }  
3     val b = async { f2() }  
4     a.await() + b.await()  
5 }
```

Kotlin, Swift, Trio

选范式 ≈ 选「错误传播 + 取消语义」

await 走异常, channel 走信号值, 结构化并发自动 cancel。

语言	关键字	类型	调度器	版本	函数颜色?
C++20	co_await	无栈/非对称	用户自带	2020	是
Go	go, chan	有栈 /对称	GMP 内建	2012	否
Python	async/await	无栈/非对称	单线程循环	2015	是
C#	async/await	无栈/非对称	线程池	2012	是
Java 21	Virtual Thread	有栈 /对称	ForkJoin	2023	否
Rust	async/await	无栈/非对称	第三方 (tokio)	2019	是

「函数颜色」问题

async 函数和普通函数互不兼容。**Go 和 Java 21 通过有栈协程把它消除了**——这是两家的核心卖点。

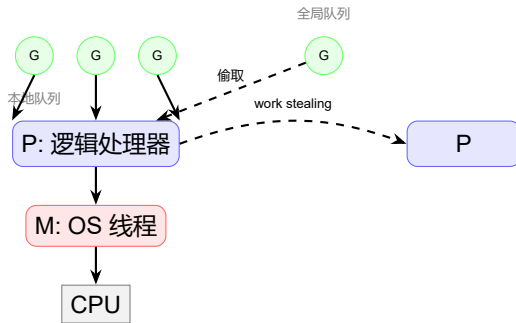
C++20: 标准只给「机制」, 运行时全靠你

C++20 协程最小示例

```
struct Task {  
    struct promise_type {  
        Task get_return_object() { return {}; }  
        std::suspend_never initial_suspend() { return {}; }  
        std::suspend_always final_suspend() noexcept { return {}; }  
        void return_void() {}  
        void unhandled_exception() {}  
    };  
};  
Task demo() {  
    std::cout << "hello ";  
    co_await std::suspend_always{};  
    std::cout << "world";  
}
```

- 编译器看到 co_await/co_yield/co_return → 函数改写为状态机
- 自动生成 frame、resume()、destroy()
- **标准库没有调度器、没有 Task、没有 IO**: 需自己写或用 cppcoro/libcoro/asio

Go: GMP 调度器 (最成功的有栈协程)



- **G** = goroutine (2KB 起步栈)
- **M** = 内核线程 (GOMAXPROCS)
- **P** = 调度上下文

- 网络 poller 集成 epoll/kqueue/IOCP
- Go 1.14+ 基于信号 SIGURG 的**抢占式调度**

Java 21 虚拟线程：「同步语法 + 协程性能」

百万并发虚拟线程

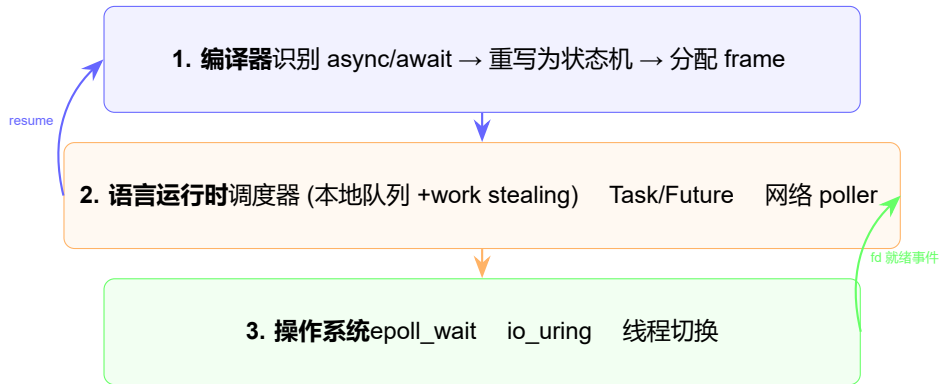
```
try (var exec = Executors.newVirtualThreadPerTaskExecutor()) {  
    for (int i = 0; i < 1_000_000; i++)  
        exec.submit(() -> { var r = http.send(req); return r; });  
}
```

- **Virtual Thread = JVM 管理的有栈协程** (Continuation + Scheduler)
- 跑在少量 carrier 平台线程上，由 ForkJoinPool 调度
- JDK 内所有阻塞 API 已改造为 yield 而非 block
- **最大卖点**：旧业务代码一行不改，Thread 换 Thread.ofVirtual() 就能扩到百万并发

Pinning 陷阱

synchronized 块、JNI 调用会把虚拟线程钉在 carrier 上

三方协同：编译器·运行时·操作系统



层	干什么	关键点
编译器	把同步代码改写为可暂停	状态机 + frame 布局
运行时	调度协程、管理 Task、对接 I/O	work stealing、IO poller
操作系统	提供非阻塞 I/O 多路复用	epoll / kqueue / io_uring / IOCP

目标：在相同硬件、相同负载下，对比三种模型的**吞吐 / 内存 / 切换成本**。

测试用例

- ❶ **I/O 密集：**10000 个并发任务，每个 sleep(0.1s)
- ❷ **CPU 密集：**10000 个并发任务，斐波那契 (n=25)
- ❸ **上下文切换：**百万次 yield/resume

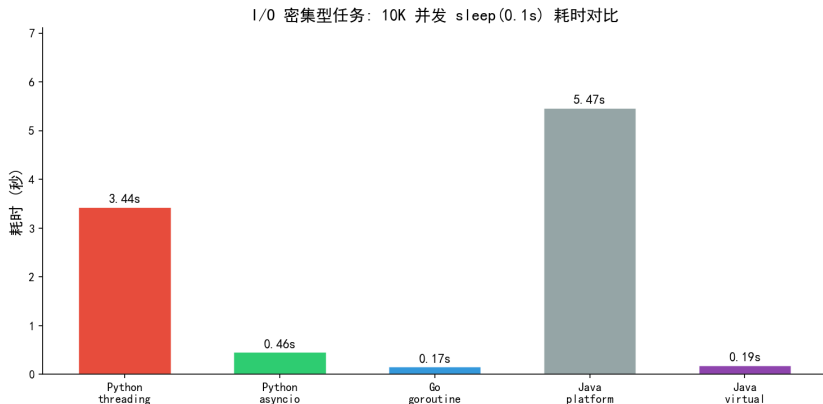
对比对象

- Python: threading vs asyncio
- Go: goroutine (GMP 调度器)
- Java: 平台线程池 vs Virtual Thread

指标

吞吐 (req/s)、P99 延迟、RSS 内存、CPU 时间

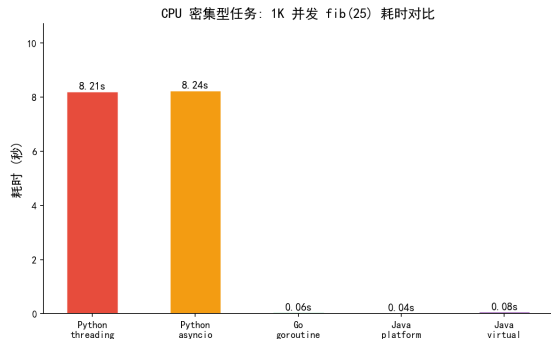
实验结果：I/O 密集型



读图方式：协程/虚拟线程不是让 sleep 更短，而是让 10000 个等待任务不占用 10000 个内核线程。

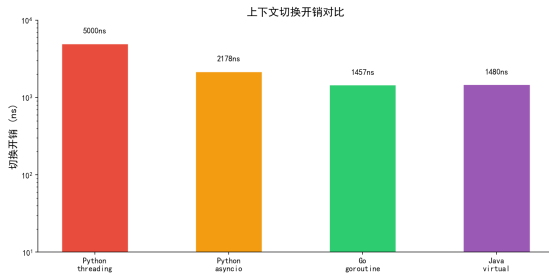
实验结果：CPU 密集型与上下文切换

CPU 密集型 (fib(25) x1000)



结论：CPU 密集时协程本身无优势，多核调度/线程池才是关键

上下文切换开销 (ns/次)



注：各语言基准语义不完全等价，只比较量级

关键发现

- **I/O 密集**: 协程/虚拟线程优势显著
- **高并发等待**: 少量内核线程承载大量逻辑任务
- **CPU 密集**: 协程本身**不会加速计算**

三个陷阱

陷阱 1: 阻塞调用毒化

在 async 里调用同步 I/O, 会卡住事件循环或 carrier。

陷阱 2: 未结构化并发

协程泄漏会变成资源泄漏; 优先使用 TaskGroup/errgroup。

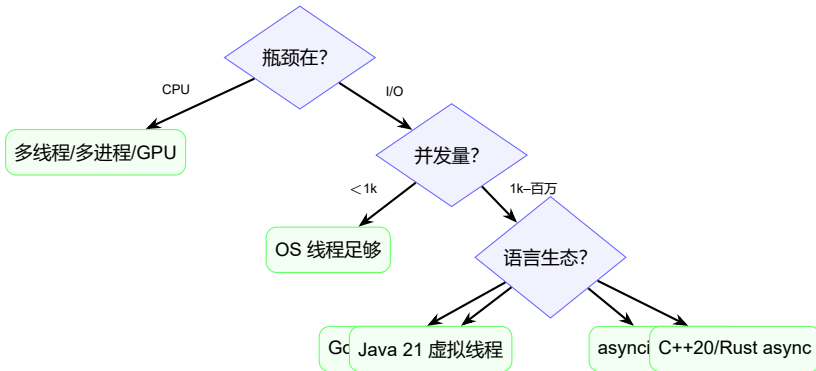
陷阱 3: 误用做 CPU 加速

协程只省等待和切换; CPU 瓶颈请上多核/SIMD/GPU。

实验口径与局限

- **I/O 测试**：10000 个 `sleep(0.1s)` 任务，模拟大量请求等待；它衡量的是调度和等待管理，不是网络协议栈真实性能。
- **CPU 测试**：`fib(25)` 用来展示“协程不自动加速计算”；Go/Java 的优势来自多核调度，不来自协程语义本身。
- **切换测试**：Python `asyncio`、Go `channel`、Java `sleep(0)` 的操作不完全同构，适合看数量级，不适合做精确排名。
- **内存测试**：Python RSS 为脚本记录值，跨语言峰值 RSS 未统一采样；最终结论以耗时和机制解释为主。

选型决策树



想**最少改代码** → Java 21 虚拟线程 / Go

想**最快上手** → Python asyncio + FastAPI
tokio / C++20 + io_uring

想**极致性能** → Rust +

协程到底带来了什么？

- **思想**：把手写的事件循环交给编译器 + 运行时
- **技术栈**：编译器（状态机）+ 运行时（调度器 + IO poller）+ OS（epoll/io_uring）
- **效果**：用线性代码扛百万并发 I/O

趋势

Java 21 虚拟线程 + Rust 异步生态 + io_uring，三者会把协程进一步「无感化」。

协程不是新东西（1958 年 Conway 就提了），
但它在 2010s–2020s 复兴，是因为：
云时代的并发量级 × 多核硬件 × 编译器技术成熟，三者同时到位。

- ❶ **信息搜集**：用 ChatGPT / Notion AI 检索 C++20 协程标准、Go GMP 调度器、Java 21 JEP-444、C# 状态机生成等技术细节，作为一手资料的「索引」。
- ❷ **结构梳理**：用 AI 列出「演进—原理—语言—实验—总结」骨架，再人工调整顺序和侧重。
- ❸ **图示生成**：TikZ 图由本人手写并让 AI 校正语法（**非截图**）。
- ❹ **实验代码**：AI 给出基准代码雏形 → 我在本机运行、修正、补齐计时与内存采集 → 真实数据替换参考量级。
- ❺ **文字润色**：所有幻灯片中文字均由本人撰写，AI 仅做术语校对和长句拆分。
- ❻ **核查**：对每条「数字 / 版本号 / 论断」均回查官方文档/JEP/PEP，避免幻觉。

标准 / 提案

- ISO C++20: [dcl.fct.def.coroutine]
- JEP 444: Virtual Threads (Java 21)
- PEP 492 / 525 / 530: Python coroutines & async generators

经典文献

- Conway M. *Design of a Separable Transition-Diagram Compiler*, 1963
- Moura & Ierusalimschy *Revisiting Coroutines*, TOPLAS 2009

运行时实现

- Go runtime source: runtime/proc.go (GMP)
- OpenJDK Loom 项目 wiki
- libuv / tokio / asio 设计文档

必读博文

- *What Color is Your Function* —Bob Nystrom
- *Why goroutines are not lightweight threads* —Dave Cheney

Q&A

欢迎提问·谢谢聆听

演示文件 + 实验代码